The Relay — Technical Report

1 What I built and why

I built **The Relay**, an async Python RAG pipeline answering engineering queries exclusively from three exact arXiv versions: *Attention Is All You Need* (`1706.03762v7`), *Moshi* (`2410.00037v1`), and *Dynamic Chunking* (`2507.07955v2`). It runs 4 steps with a vector tool call and an early-refusal branch: (1) **Query Understanding** validates length and domain keywords; (2) **Retrieval Engine** queries persistent ChromaDB (`all-MiniLM-L6-v2` embeddings; optional `cross-encoder/ms-marco-MiniLM-L-6-v2` reranker) and attaches `[Paper: <Title> (<id>), Page <p>]` citations; (3) **Answer Generation** dispatches prompts through `AsyncRateLimiter` and `RetryManager` to Groq or the stand-in simulator; (4) **Citation & Quality Checker** regex-verifies citations/refusal phrases to grade `correct`, `wrong`, `refused_correctly`, or `refused_wrongly`.

Decision that shaped it most: Early refusal branching at Step 2 (`max_similarity < 0.30`). Out-of-domain queries scored ~0.11 vs. ~0.65 for in-domain queries. Short-circuiting below `0.30` returns an explicit refusal in **14–20 ms** at **0 LLM tokens**, saving scarce token-per-minute quota and preventing hallucination under burst traffic.

2 The real service and my stand-in

I probed Groq's free tier (`openai/gpt-oss-20b`, after discovering `llama-3.1-8b-instant` returned 404 via `/models`) and read live `x-ratelimit-*` headers and streaming latencies (~853 ms TTFT, ~150 tok/s output).

Table 1: Limits you found in the real service

Limit	Value you found	How you found it	Confidence
Requests per minute	1,000 RPM	`x-ratelimit-limit-requests` response header	High
Tokens per minute	8,000 TPM	`x-ratelimit-limit-tokens` response header	High
Requests per day	14,400 RPD	Groq platform docs & console quota	High
Tokens per day	500,000 TPD	Groq free-tier quota docs	Medium
Requests in flight at once	1–3 concurrent (inferred)	Sequential high-TPM runs throttled; hard cap not isolated	Low
What it returns when a limit is hit	HTTP 429 (`rate_limit_exceeded`) + `retry-after`	Groq API error docs & SDK `RateLimitError` handler	High
Time until it recovers	577ms (tokens), 1m 26.4s (requests)	`x-ratelimit-reset-tokens` / `reset-requests` headers	High

Stand-in calibration & differences: `SimulatedModelService` anchors baseline TTFT at 850 ms (±jitter) and token streaming at 7 ms/word (~143 tok/s), adding concurrency contention (`<=2` in-flight: ~0.85s; `3-5`: 2.8–4.6s; `>5`: up to 11s), 2% baseline HTTP 500 errors (scaling with load), sliding-window RPM/daily 429s, and timeout triggers. **Where it differs most:** Real Groq runs on multi-tenant LPU hardware constrained tightly by continuous **TPM** refills (577 ms) across shared traffic, whereas our stand-in models **RPM**, daily count, and local asyncio concurrency without token-level bucket depletion or network jitter.

3 Results

All 268 traces in `results.csv` cover **real Groq** (48 retrieval queries across 16 self-written questions × 3 settings, paced 2s apart) and the **stand-in** (220 requests across 5 load scenarios). In Table 2a, *Duration* is wall-clock completion time and *Offered rate* is `requests / duration * 60` (note: `burst` launched all 50 tasks at `t=0s`). Percentiles report median (`p50`) and slow tail (`p95`, `p99`).

Table 2a: Latency, one row per load scenario

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady	stand-in	20	30.12	39.8	863.4 / 936.5 / 941.2	1335.7 / 1436.2 / 1641.6
burst	stand-in	50	113.36	26.5	8000.0 / 8000.0 / 8000.0	64172.4 / 112025.8 / 112770.2
sustained_overload	stand-in	80	134.01	35.8	6323.1 / 8000.0 / 8000.0	24763.6 / 124502.9 / 128547.3
rpm_halved	stand-in	50	44.27	67.8	0.0 / 8000.0 / 8000.0	21036.7 / 34551.6 / 34862.8
daily_budget_exhausted	stand-in	20	3.08	389.6	0.0 / 0.0 / 0.0	852.8 / 1153.0 / 1192.6

Table 2b: Outcomes, same rows

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
steady	100.0%	0.0%	0.0%	0.00	1	1068.6	$0.0546
burst	20.0%	0.0%	80.0%	1.70	40	1067.4	$0.0545
sustained_overload	8.8%	41.2%	50.0%	1.04	35	1081.1	$0.0552
rpm_halved	10.0%	80.0%	10.0%	1.80	35	1094.4	$0.0559
daily_budget_exhausted	0.0%	100.0%	0.0%	2.00	1	0.0	$0.0000

Number I couldn't have guessed: In `burst` (50 concurrent tasks, 10 semaphore slots, 1,000 RPM limit), **peak queue hit 40 and 80.0% timed out** with p95 total latency reaching **112,025.8 ms** because waiting behind slow 8s retries starved queued requests before execution. On the 16 real-Groq evaluation questions, `setting_a` achieved **93.8% overall accuracy** (68.8% correct+cited, 25.0% correctly refused, 0.0% wrong) at **1,009.2 tokens/answer**.

4 Where the time goes

Tracing **`req_0fe76a5a`** (`retrieval_setting_c`, `real` Groq, `total_ms = 31837.87`, `first_token_ms = 10000.00`, `retries = 2`, `status = timeout`):

- **Waiting (local queue):** 0 ms (sequential run; no semaphore contention).
- **Working (3 provider waits + local retrieval): 30,000 ms** blocked on 3 Groq attempts (`10.0s` client deadline each) plus local Chroma/CrossEncoder reranking overhead.
- **Retrying (backoff sleep): 1,837.87 ms** combined across 2 jittered backoff sleeps (`Uniform(0.1, 0.5s)` + `Uniform(0.1, 1.0s)`) and local retrieval/rerank overhead (only aggregate `total_ms` is stored in CSV).

Before or after first token: Before. Because `first_token_ms` equals the exact `10,000.0 ms` timeout in `results.csv` and `tokens_out = 0`, Groq throttled prefill under TPM saturation before streaming a single byte. *(In `burst`, `req_c397436c` similarly took `112,914.98 ms` total while `first_token_ms` was `4,169.39 ms`—proving ~100s was spent waiting in the local semaphore queue before attempt 2 succeeded).*

5 When a limit is hit

- **Per-minute limit:** `AsyncRateLimiter` caps concurrent slots (8–10) and paces RPM via a 60s sliding window (sleeping up to 1s). In `sustained_overload`, once queue depth reaches `max_queue_depth = 35`, new arrivals are **dropped** (`41.2% rate_limited`). In `rpm_halved` (50% cut to 30 RPM), 80.0% hit 429s.
- **Per-day limit:** Requests are rejected by the provider with 429 (`daily_budget_exhausted`: 0% succeeded, 0 tokens billed).
- **Who is delayed, dropped, or gets a weaker answer:** Queued requests within depth 35 are **delayed** (FIFO semaphore wait); excess requests over depth 35 (or exceeding 8s deadlines) are **dropped**; unanswerable queries get instant refusals, while shifting to `setting_a` (or dropping `K=5 -> 2` chunks when close to the 8k TPM cap) trades cross-passage breadth for staying live, yielding a **weaker/thinner context** rather than timing out.

Retrieval: quality versus cost

Table 3: Retrieval settings

Setting (piece size, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
300 words, 3 chunks, No	16	68.8%	0.0%	25.0%	1009.2	1296.5 / 9093.4	3125.1 / 14707.9
500 words, 5 chunks, No	16	25.0%	43.8%	25.0%	1314.6	7659.6 / 10000.0	27692.5 / 31115.6
700 words, 8 chunks, Yes	16	37.5%	43.8%	18.8%	2115.1	4600.6 / 10000.0	17752.5 / 35669.8

Choice & limit-aware policy: I chose **Setting A (300w, 3 chunks, no rerank)** because it yielded **0.0% wrong answers** and **93.8% valid accuracy** at 1,009 tokens/answer. Settings B and C inflated prompts to 3.5k–5.7k tokens, breaching Groq's 8,000 TPM cap on back-to-back calls and causing 43.8% timeouts. When close to TPM limits, retrieval should clamp `top_k` from 3 to 2 and truncate chunks to keep total prompt size < 800 tokens.

How this library will mislead a naive retriever

1. **Terminology collision ("base model"):** Query *"What is the hidden dimension of the base model?"* ranked *Dynamic Chunking* p. 4 (`0.4242`) and p. 16 (`0.4156`, DNA **"base pairs"** A,C,G,T!) above *Attention* p. 9 Table 3 (`0.3494`). A naive top-2 retriever completely misses the Transformer paper.
2. **Math notation blindness:** Query *"What does PE_(pos,2i) = sin(pos / 10000^(2i/d_model)) mean?"* scored `0.2907` on *Dynamic Chunking* p. 4 vs `0.2624` on *Attention* p. 9—falling below `0.30` and missing Eq. 3 on p. 6. In our real test (`q2_attn`), PDF equation fragmentation caused `refused_wrongly` on both Settings A and B.
3. **Keyword dominance over intent:** Query *"How are tokens processed in streaming chunks?"* (about Moshi's real-time audio) retrieved only *Dynamic Chunking* pp. 31, 38, 34 (`0.475–0.452`) and zero *Moshi* passages because "tokens"+"chunks" drowned out "streaming".

Your feedback on Vaani

Visiting `vaani.dev`, the emphasis on sub-500ms multilingual voice AI agents and phone-number deployment harnesses stood out clearly. What confused me was the lack of concrete latency breakdowns separating ASR, LLM, and TTS overhead on the landing flow. I would add a live interactive TTFT/end-to-end latency demo widget with network throttling toggles directly on the homepage.

6 Retries

Policy: Up to `max_retries = 2` on `rate_limited`, `error`, or `timeout` (`10s` retrieval / `8s` load tests) using full-jitter exponential backoff: `Uniform(0.1, min(4.0, 0.5 * 2^attempt))`. **Where retrying made things worse:** In `daily_budget_exhausted` (and Settings B/C TPM saturation), `RetryManager` blindly retried `rate_limited` responses twice. For `daily_budget_exhausted`, this multiplied load 3× against a 0-quota provider and delayed failure from ~15 ms to **852.8 ms p50**; under `burst`, 1.7 retries/request kept worker slots blocked ~24s per failing job, starving queued users.

7 What the person sees

At 3 PM when daily budget is gone, our prototype exhausts 2 retries (~853 ms) and returns `status="rate_limited"` (`error="429: Daily request quota exhausted"`). In production, `RetryManager` must **skip retries on daily-quota errors** and immediately return an explicit user message: *"The Relay has reached its daily research capacity (resets at 00:00 UTC). Please retry tomorrow or contact the platform owner for urgent lookups."*

8 What I'm unsure about, what I left out, what I learned

- **Least certain assumption:** Estimating tokens via `words * 4 // 3` and hardcoding `0.30` cosine threshold across all chunk sizes. Dense math/code doubles token counts (risking unexpected 8k TPM hits), and larger chunks dilute similarity scores (`setting_c` got 3/4 refusals right vs 4/4).
- **Two things deliberately left out:** (1) Automatic dynamic `K` shedding wired to `x-ratelimit-remaining-tokens` headers; (2) A Redis distributed queue (unnecessary network complexity for single-node asyncio benchmarks).
- **One thing new to me: TPM dominance over RPM.** In RAG apps, large retrieved contexts silently blow through Token-Per-Minute ceilings (8k TPM = ~1.5 Setting C calls/min) while Request-Per-Minute usage looks nearly idle (<1% of 1,000 RPM).

How I worked with the assistant

The verbatim session log is attached separately in `conversation_history.md`. **Three prompts that mattered most:** (1) *"dont write code into code base directly, tell me what to do and why"*—forced incremental understanding so I can defend every line; (2) *"i want to use chroma db"*—kept rich paper/page metadata coupled to vectors on disk; (3) *"i want to use 3.10 only"*—forced diagnosing Windows wheel compatibility (`onnxruntime<1.24`) instead of blindly upgrading Python. **Mistakes caught:** (1) Assistant defaulted to `llama-3.1-8b-instant` which 404'd; I probed `/models` and switched to `openai/gpt-oss-20b`; (2) `compute_tables.py` initially aggregated 220 load-test rows into Table 3 Setting A (`236` questions); I caught it and fixed the filter to `scenario == f"retrieval_{set_key}"`; (3) Assistant's initial draft cited a fabricated `req_3183787` ID, which we caught and replaced with real `req_0fe76a5a`. **Overruled:** Rejected FAISS for ChromaDB and rejected upgrading Python 3.10 to 3.11. **Distinctly mine:** The 16-question cross-paper test suite, `uv` + Python 3.10 dependency resolution, catching the 8k TPM bottleneck, and testing the 3 real retriever failure traps.